

UNIVERSIDAD DE SAN CARLOS DE GUATEMALA
CENTRO UNIVERSITARIO DEL NORTE

-CUNOR-

“Entregable 3”



Docente:

Ing. José Alberto Veliz Cruz

Curso:

Lenguajes Formales Y De Programación

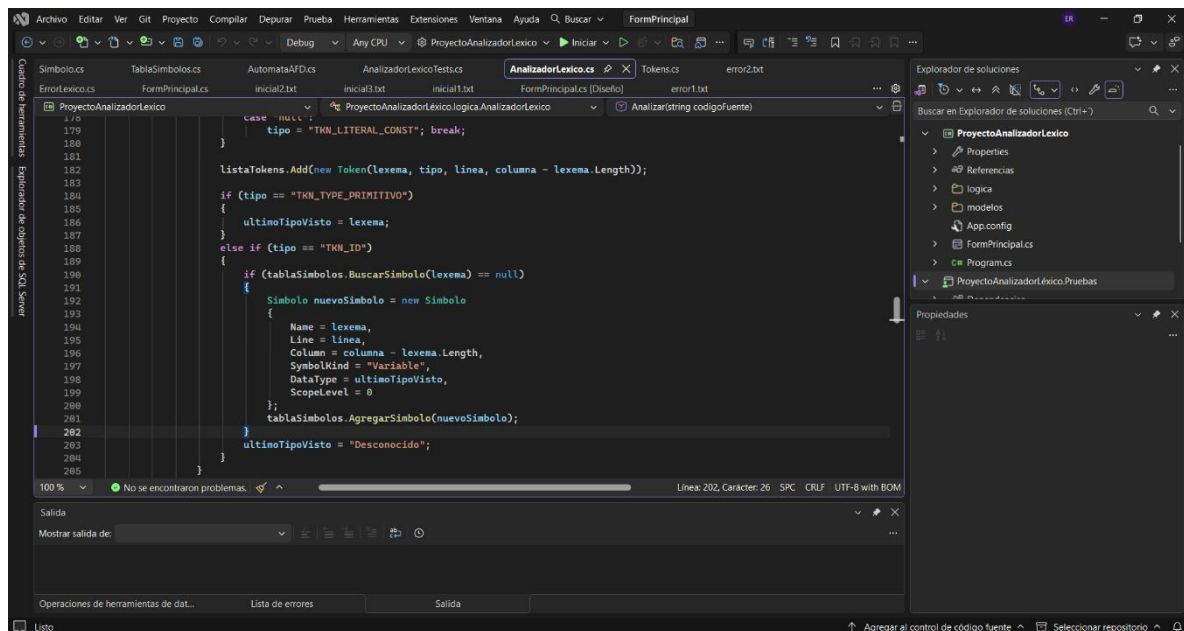
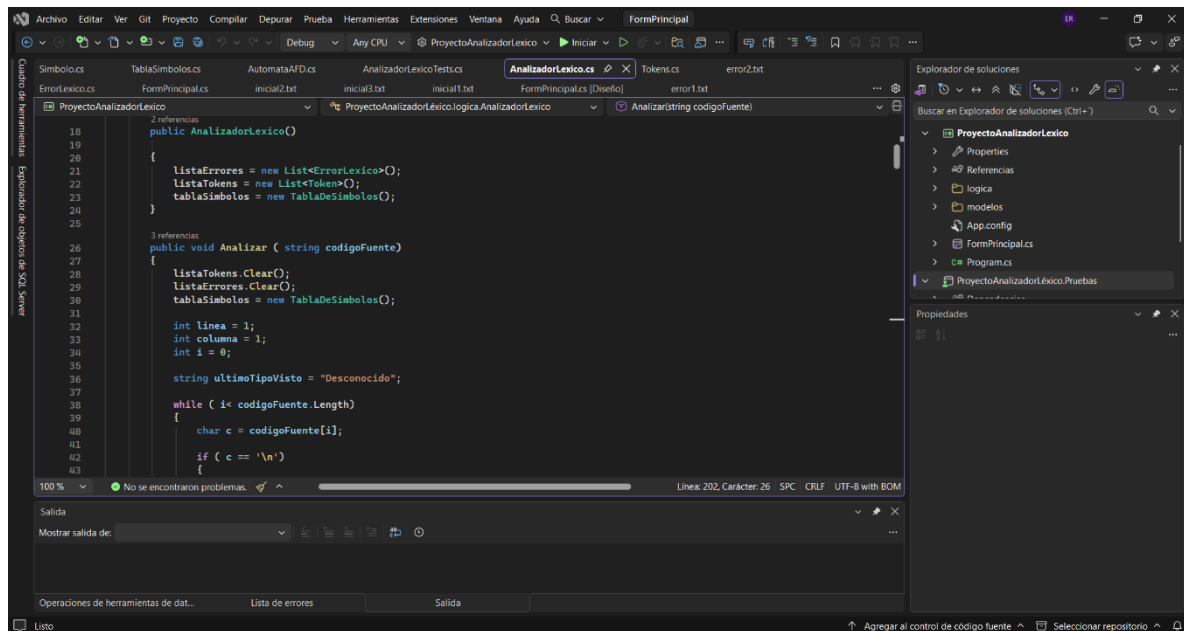
Estudiantes:

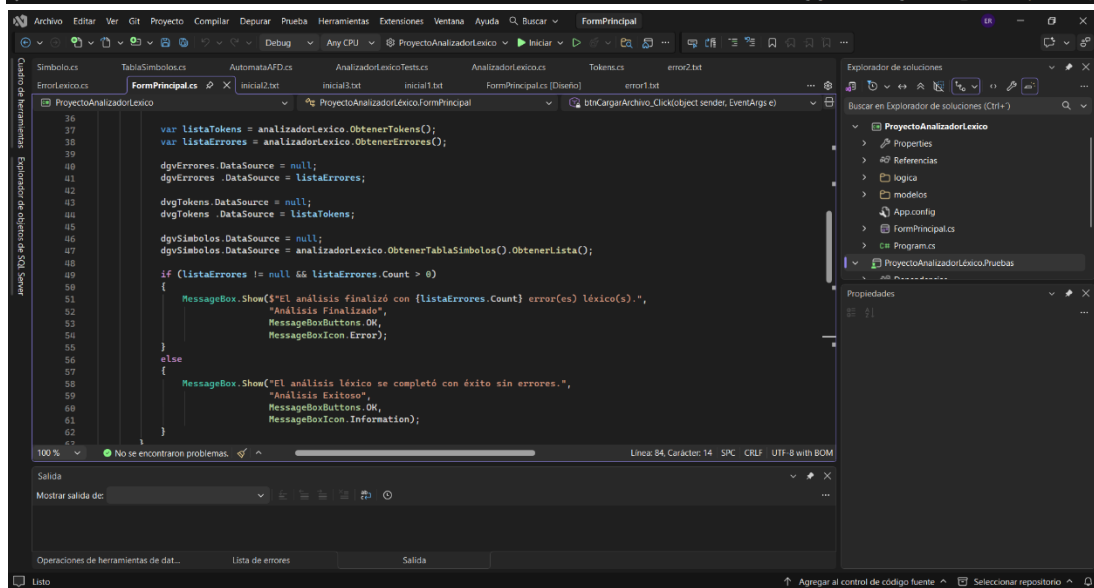
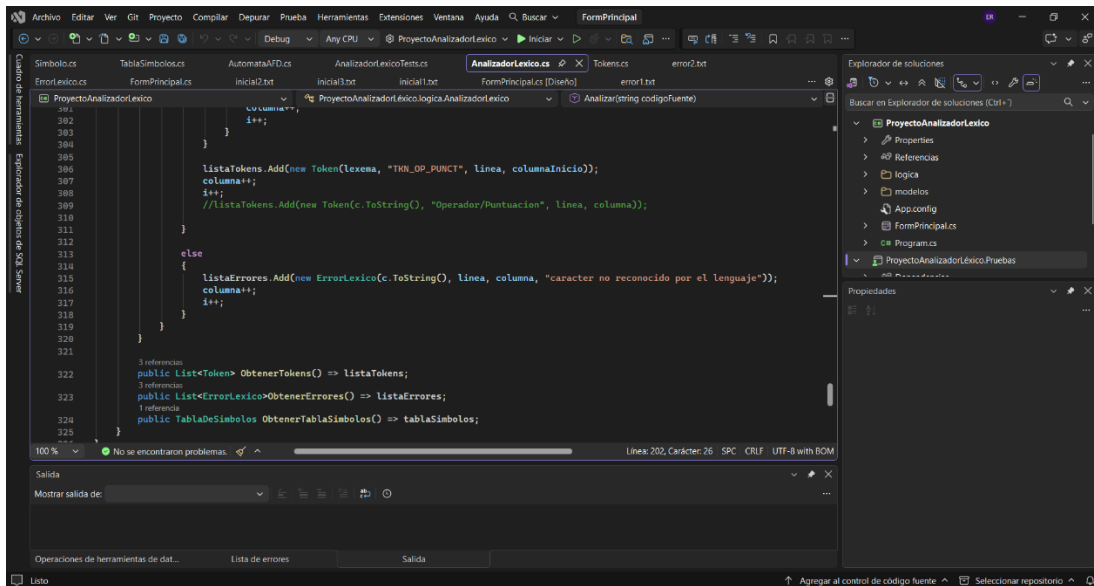
Verónica Estefanía Reinoso Castellanos	ICS	202542900
Hossellyn Victoria Asucena Morán Cojoc	ICS	202440820
Brandon Iván Caal Jul	ICS	202441122

Cobán, Alta Verapaz, Septiembre 2026

- **Verónica Estefanía Reinoso Castellanos - 202542900**

En esta entrega se integró la visualización de la Tabla de Símbolos en la interfaz gráfica del analizador léxico. Se implementó la lógica para reconocer y asociar el tipo de dato primitivo (DataType) a los identificadores declarados. También se realizaron las conexiones necesarias en el formulario principal para proyectar correctamente la información analizada dentro del DataGridView de símbolos (dgvSimbolos).





Form1

Editor de Código Fuente:

```

int contador = 0;
float promedio = 15.5;
string mensaje = "Inicio de proceso";
if (contador < 10) {
    contador = contador + 1;
}

```

Tokens:

	Tipo	Valor	Linea	Columna
▶	TKN_TYPE_P...	int	1	1
	TKN_ID	contador	1	5
	TKN_OP_PU...	=	1	14
	TKN_NUM_INT	0	1	16

Simbolos

	Name	SymbolKind	Data Type	ScopeLevel
▶	contador	Variable	int	0
	promedio	Variable	float	0
	mensaje	Variable	string	0

Errores:

	CaracterCadena	Linea	Columna	Descripcion
--	----------------	-------	---------	-------------

Archivo

Analizar

- Brandon Iván Caal Jul- 202441122

```

18
19     10 referencias
20     public class Token
21     {
22         1 referencia
23         public string Tipo { get; set; }
24         1 referencia
25         public string Valor { get; set; }
26         1 referencia
27         public int Linea { get; set; }
28         1 referencia
29         public int Columna { get; set; }
30
31         5 referencias
32         public Token(string lexema, string tipo, int linea, int columna)
33         {
34             Valor = lexema;
35             Tipo = tipo;
36             Linea = linea;
37             Columna = columna;
38         }
39     }

```

Para cumplir con el reconocimiento de múltiples categorías, en el apartado tokens.cs, se estableció una estructura de datos base. Esta clase actúa como el molde que recibe cada fragmento de texto puro y le asocia permanentemente su categoría, además de registrar sus líneas y columnas.

```

if (char.IsLetter(c) || c == '_')
{
    string lexema = "";
    while (i < codigoFuente.Length && (char.IsLetterOrDigit(codigoFuente[i]) || codigoFuente[i] == '_'))
    {
        lexema += codigoFuente[i];
        columna++;
        i++;
    }

    //listaTokens.Add(new Token(lexema, "Identificador", linea, columna - lexema.Length));

    string tipo = "TKN_ID";
    switch (lexema)
    {
        // Tipos integrados / primitivos
        case "bool":
        case "byte":
        case "char":
        case "decimal":
        case "double":
        case "float":
        case "int":
        case "long":
        case "object":
        case "sbyte":
        case "short":
        case "string":
        case "uint":
        case "ulong":
        case "ushort":
        case "void":
            tipo = "TKN_TYPE_PRIMITIVO"; break;

        // Declaración de tipos
        case "class":
        case "struct":
        case "interface":
        case "enum":
        case "delegate":
        case "record":
            tipo = "TKN_TYPE_DECL"; break;
    }
}

```

```

    case "record":
        tipo = "TKN_TYPE_DECL"; break;

    // Modificadores de acceso
    case "public":
    case "private":
    case "protected":
    case "internal":
    case "file":
        tipo = "TKN_MOD_ACCESO"; break;

    // Modificadores de comportamiento
    case "abstract":
    case "async":
    case "const":
    case "event":
    case "extern":
    case "in":
    case "out":
    case "override":
    case "readonly":
    case "ref":
    case "sealed":
    case "static":
    case "virtual":
    case "volatile":
        tipo = "TKN_MOD_COMP"; break;

```

```

// Sentencias de control y bucles
case "break":
case "case":
case "continue":
case "default":
case "do":
case "else":
case "for":
case "foreach":
case "goto":
case "if":
case "return":
case "switch":
case "while":
    tipo = "TKN_CTRL_BCLS"; break;

// Manejo de excepciones
case "try":
case "catch":
case "finally":
case "throw":
    tipo = "TKN_EXCEPT_DECL"; break;

// Gestión de memoria y punteros
case "fixed":
case "stackalloc":
case "unsafe":
    tipo = "TKN_MEM_DECL"; break;

```

```

    tipo = "TKN_MEM_DECL"; break;

    // Operadores e Inspección de Tipos
    case "as":
    case "await":
    case "checked":
    case "unchecked":
    case "is":
    case "nameof":
    case "typeof":
    case "new":
    case "sizeof":
        tipo = "TKN_KEYWORD_OP"; break;

    // Ámbitos y referencias de instancia
    case "base":
    case "this":
    case "namespace":
    case "using":
    case "lock":
        tipo = "TKN_SCOPE_REF"; break;

    // Literales booleanos y nulos
    case "false":
    case "true":
    case "null":
        tipo = "TKN_LITERAL_CONST"; break;
}

ListaTokens.Add(new Token(lexema, tipo, linea, columna - lexema.Length));

```

El autómata lee secuencias de letras y asume inicialmente que se trata de un identificador creado por el usuario (TKN_ID). Sin embargo, antes de almacenarlo, el lexema atraviesa una estructura switch que funciona como un diccionario del lenguaje C#. Esto permite recategorizar palabras clave específicas en sus subcategorías correctas

```

    }
    else if (char.IsDigit(c))
    {
        string lexema = "";
        bool tienePunto = false;

        while (i < codigoFuente.Length && (char.IsDigit(codigoFuente[i]) || codigoFuente[i] == '.'))
        {
            if (codigoFuente[i] == '.')
            {
                if (tienePunto) break;
                tienePunto = true;
            }

            lexema += codigoFuente[i];
            columna++;
            i++;
        }
        string tipoToken = tienePunto ? "TKN_NUM_DEC" : "TKN_NUM_INT";

        listaTokens.Add(new Token(lexema, tipoToken, linea, columna - lexema.Length));
    }
}

```

```

    else if (c == '"')
    {
        string lexema = c.ToString();
        int columnaInicio = columna;
        columna++;
        i++;

        while (i < codigoFuente.Length && codigoFuente[i] != '"' && codigoFuente[i] != '\n')
        {
            lexema += codigoFuente[i];
            columna++;
            i++;
        }

        if (i < codigoFuente.Length && codigoFuente[i] == '"')
        {
            lexema += codigoFuente[i];
            columna++;
            i++;
            listaTokens.Add(new Token(lexema, "TKN_CADENA_TEXTO", linea, columnaInicio));
        }
        else
        {
            listaErrores.Add(new ErrorLexico(lexema, linea, columnaInicio, "Cadena de texto sin cerrar"));
        }
    }
}

```

Se añadieron validaciones para datos primitivos. El bloque numérico inspecciona la presencia de un punto decimal para asignar la categoría de número entero o número decimal. Adicionalmente, el código cuenta con ramas de ejecución paralelas para encapsular secuencias de texto entre comillas dobles y simples, cubriendo las categorías de literales de cadena y carácter.

```

else if (c == '=' || c == '+' || c == '-' || c == ';' || c == '*' || c == '/' ||
c == '!' || c == '&' || c == '|' || c == '.' || c == ',' ||
c == '{' || c == '}' || c == '(' || c == ')' || c == '[' || c == ']' ||
c == '<' || c == '>')
{
    string lexema = c.ToString();
    int columnaInicio = columna;

    if (i + 1 < codigoFuente.Length)
    {
        char siguiente = codigoFuente[i + 1];
        if ((c == '=' && siguiente == '=') ||
            (c == '!' && siguiente == '=') ||
            (c == '&' && siguiente == '&') ||
            (c == '|' && siguiente == '|') ||
            (c == '+' && siguiente == '+') ||
            (c == '-' && siguiente == '-') ||
            (c == '<' && siguiente == '=') ||
            (c == '>' && siguiente == '='))
        {
            lexema += siguiente;
            columna++;
            i++;
        }

        listaTokens.Add(new Token(lexema, "TKN_OP_PUNCT", linea, columnaInicio));
        columna++;
        i++;
    }
}

```

Para reconocer la categoría de operadores y delimitadores de manera precisa, se añadió la técnica de inspección de un carácter a futuro. Esto permite que el escáner no solo capture operadores de un solo símbolo, sino que evite errores lógicos agrupando correctamente operadores compuestos, clasificándolos con exactitud bajo la categoría general de puntuación y operadores.

```

6  using System.Threading.Tasks;
7
8  namespace ProyectoAnalizadorLéxico.modelos
9  {
10     7 referencias
11     public class Simbolo
12     {
13         3 referencias
14         public string Name { get; set; }
15         1 referencia
16         public string SymbolKind { get; set; }
17         1 referencia
18         public string DataType { get; set; }
19         1 referencia
20         public int ScopeLevel { get; set; }
21         1 referencia
22         public int Line { get; set; }
23         1 referencia
24         public int Column { get; set; }
25     }
26 }

```

Para cumplir con el registro detallado de cada identificador, En símbolo.cs, se creó un modelo de datos específico separado de los tokens generales. Esta clase define los atributos semánticos de una variable definida por el usuario.

```

namespace ProyectoAnalizadorLéxico.logica
{
    4 referencias
    public class TablaDeSimbolos
    {
        5 referencias
        private Dictionary<string, Simbolo> simbolos = new Dictionary<string, Simbolo>();

        1 referencia
        public bool AgregarSimbolo(Simbolo s)
        {
            if (!simbolos.ContainsKey(s.Name))
            {
                simbolos.Add(s.Name, s);
                return true;
            }

            return false;
        }

        1 referencia
        public Simbolo BuscarSimbolo(string nombre)
        {
            if (simbolos.ContainsKey(nombre))
            {
                return simbolos[nombre];
            }

            return null;
        }

        1 referencia
        public List<Simbolo> ObtenerLista()
        {
            return simbolos.Values.ToList();
        }
    }
}

```

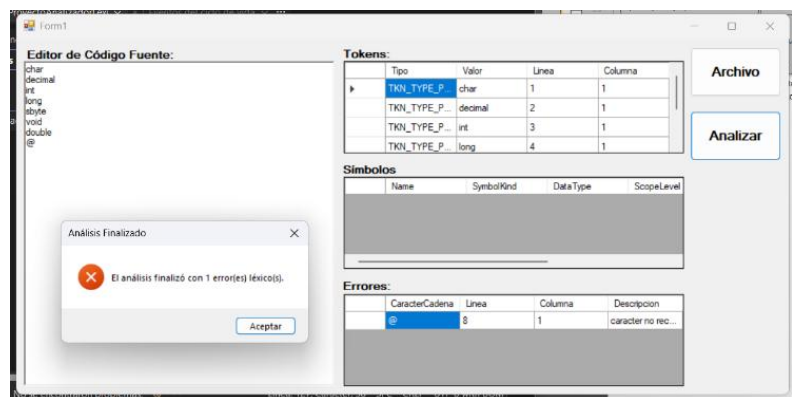
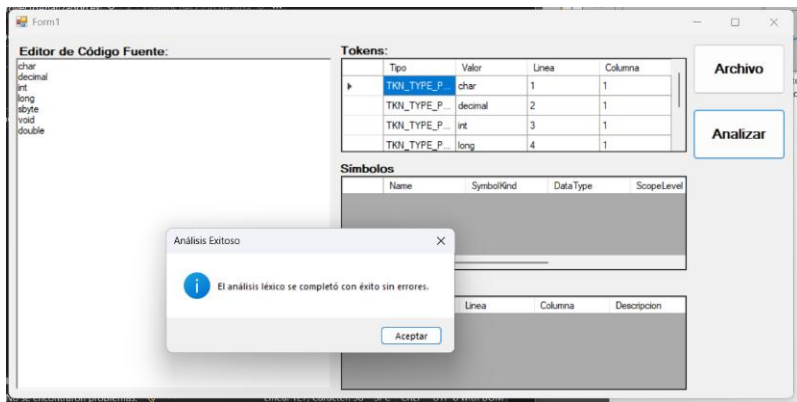
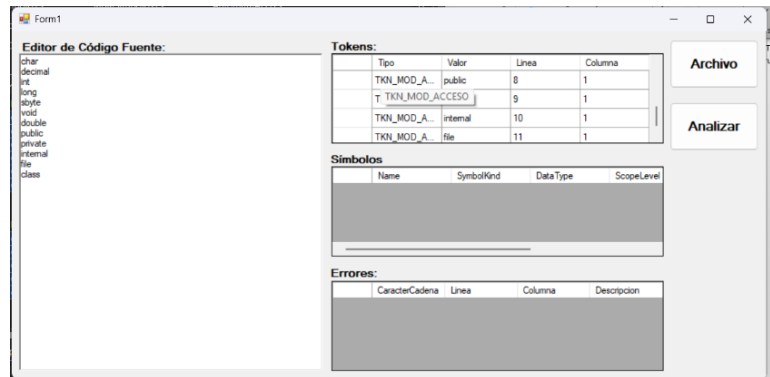
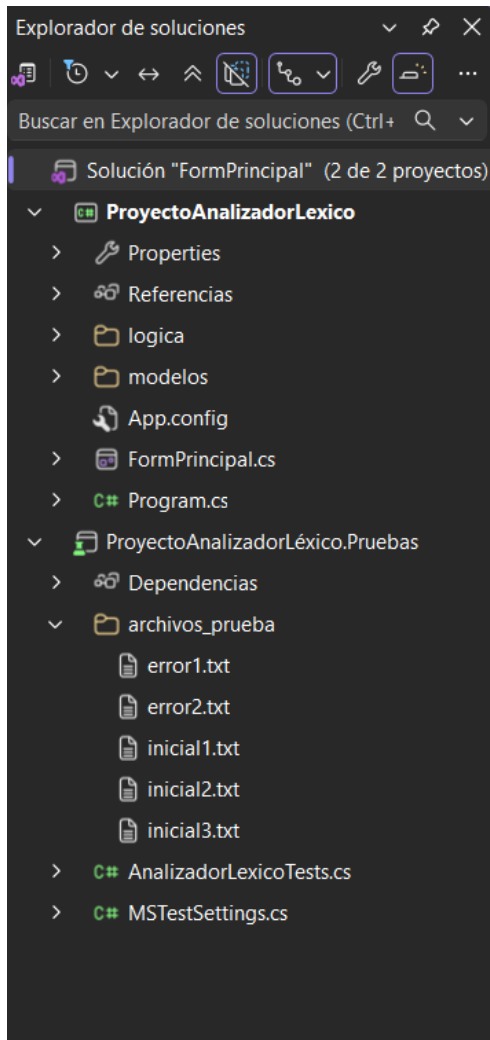
En TablaSimbolos.cs, este elemento es el encargado de evitar duplicados. En lugar de utilizar una lista secuencial, la Tabla de Símbolos se implementó sobre un Dictionary<string, Simbolo>. El método AgregarSimbolo valida la preexistencia del identificador usando ContainsKey. Al rechazar cualquier inserción si el nombre ya existe, se garantiza que no haya variables repetidas y que los datos guardados correspondan únicamente a la primera vez que el escáner detectó dicho identificador.

```
if (tipo == "TKN_TYPE_PRIMITIVO")
{
    ultimoTipoVisto = lexema;
}
else if (tipo == "TKN_ID")
{
    if (tablaSimbolos.BuscarSimbolo(lexema) == null)
    {
        Simbolo nuevoSimbolo = new Simbolo
        {
            Name = lexema,
            Line = linea,
            Column = columna - lexema.Length,
            SymbolKind = "Variable",
            DataType = ultimoTipoVisto,
            ScopeLevel = 0
        };
        tablaSimbolos.AgregarSimbolo(nuevoSimbolo);
    }
    ultimoTipoVisto = "Desconocido";
}
}
```

Esta parte en AnalizadorLexico.cs, hace que solo cuando un token se clasifica definitivamente como un identificador de usuario TKN_ID, se procesa para la tabla de símbolos. En ese instante exacto se captura el valor de la variable línea del ciclo, cumpliendo con el registro de la primera aparición. Aparte de eso, el algoritmo utiliza una bandera de estado para asociarle su tipo de dato correspondiente.

- **Hossellyn victoria Asucena Morán Cojoc -202440820**

Para este entregable, lo que se hizo fue la detección de errores para que el analizador no se detenga al hallazgo de un error detectando el tipo de carácter/cadena, línea columna y descripción de la misma para poder continuar con el escaneo, también escribir las pruebas que en este caso se utilizó el MTses/unit , enlazado con el analizador para poder validar los tres archivos de prueba junto con los dos casos adicionales.



código del analizador para la validación de los archivos de prueba

```
FormPrincipal.cs [Diseño]  AnalizadorLexicoTests.cs  inicial3.txt  inicial1.txt  error1.txt
ProyectoAnalizadorLéxico.Pruebas
1 using Microsoft.VisualStudio.TestTools.UnitTesting;
2 using ProyectoAnalizadorLéxico.logica;
3 using System.IO;
4
5 namespace ProyectoAnalizadorLéxico.Pruebas
6 {
7     [TestClass]
8     public class PruebasAnalizador
9     {
10         private AnalizadorLexico analizador;
11
12         [TestInitialize]
13         public void Setup()
14         {
15             analizador = new AnalizadorLexico();
16         }
17
18         [TestMethod]
19         public void Validar_ArchivoInicial1_NoGeneraErrores()
20         {
21             string ruta = @"archivos_prueba\inicial1.txt";
22             string codigo = File.ReadAllText(ruta);
23
24
25             analizador.Analizar(codigo);
26             var errores = analizador.ObtenerErrores();
27             var tokens = analizador.ObtenerTokens();
28
29             Assert.AreEqual(0, errores.Count, "El archivo inicial1.txt no debería contener errores léxicos.");
30             Assert.IsTrue(tokens.Count > 0, "Se debieron generar tokens válidos.");
31         }
32     }
33 }
```

```
ormPrincipal.cs [Diseño]  AnalizadorLexicoTests.cs  inicial3.txt  inicial1.txt  error1.txt
ProyectoAnalizadorLéxico.Pruebas  ProyectoAnalizadorLéxico.Pruebas.PruebasAnalizador  Validar_ArchivoInicial
34
35     [TestMethod]
36     public void Validar_ArchivoError1_ContinuaEscaneo()
37     {
38
39         string ruta = @"archivos_prueba\error1.txt";
40         string codigo = File.ReadAllText(ruta);
41
42
43         analizador.Analizar(codigo);
44         var errores = analizador.ObtenerErrores();
45         var tokens = analizador.ObtenerTokens();
46
47         Assert.IsTrue(errores.Count > 0, "Se esperaba detectar errores en error1.txt.");
48         Assert.IsTrue(tokens.Count > 0, "El analizador se detuvo en el error y no extrajo los tokens válidos restantes.");
49     }
50 }
51
52 }
```

Link del video de Prueba: <https://www.youtube.com/watch?v=ls44sXa-6bk>